Système d'exploitation : TP7
Tubes et redirections

mars 2026 — Pierre Rousselin

Exercice 1 : Introduction aux tubes

1. Télécharger, lire, compiler et exécuter le fichier `exemple_tube1.c`.
2. Comment se sert-on de l'appel système `pipe` ? Quel descripteur de fichier est utilisé pour lire dans le tube ? pour écrire dans le tube ?
3. Remarquer qu'appuyer sur `Ctrl-D` à la fin de la saisie indique une fin de fichier pour la lecture sur le terminal et met fin au programme. Commenter la ligne qui ferme l'extrémité `1` du `tube`, recompiler et répéter l'expérience précédente. Le programme ne se termine pas. Où est le *deadlock* ?
4. Ici, le tube et le `fork` sont en fait inutiles. Pourquoi ?
5. Modifier le programme de façon à ce que le parent remplace tous les chiffres décimaux par des espaces avant d'écrire dans le tube.
6. Ajouter un deuxième enfant et un deuxième tube. Le premier enfant écrit dans le deuxième tube maintenant. Le nouvel enfant répète chaque caractère alphabétique qu'il trouve dans ce tube et écrit sur la sortie standard.

--- * ---

Exercice 2 : `ls | grep`

L'appel système `dup2` :

```
int dup2(int oldfd, int newfd);
```

permet dans les faits une redirection ; au retour de cet appel système `newfd` décrit le même fichier que `oldfd`. Par exemple après `dup2(2, 1)`, la sortie standard est redirigée vers la sortie standard des erreurs. En utilisant un tube, deux processus enfant, des redirections avec `dup2` et deux appels systèmes `execlp`, créer un programme C équivalent à la commande `ls / | grep u`. Remarque : les redirections avec `dup2` et les fichiers ouverts sont encore disponibles après `execlp`.

--- * ---

Exercice 3 : Nombre secret et nombre secret inverse

1. Télécharger et lire les deux fichiers `nb_secret.c` et `nb_secret_inverse.c`.
2. Les compiler avec des noms différents (par exemple, `nbs` et `nbsi`). Exécuter les deux programmes au moins une fois pour voir leur fonctionnement.
3. Le but est maintenant de les faire jouer ensemble. D'abord avec le shell.
   a) La commande `mkfifo mon_tube` crée un nouveau fichier (un *tube nommé*). La lancer et voir le résultat de `ls -l`. Le nouveau fichier devrait être du type `p` (comme *pipe*).
   b) La commande `tee` permet de répéter son entrée standard dans un fichier et sur sa sortie standard. Tester avec, par exemple, la commande `tee plop.txt` (entrer plusieurs lignes puis `Ctrl-D`, vérifier que l'entrée est bien recopiée sur la sortie standard et écrite dans le fichier `plop.txt`).
   c) Le dernier élément utile est la commande `stdbuf -o0 COMMANDE` qui lance `COMMANDE` avec *une sortie non bufferisée*. C'est important car en redirigeant la sortie dans un tube, on perd le côté *line buffered* du terminal et on risque le *deadlock*.
   d) Enfin, on rappelle que le fichier `/dev/tty` désigne toujours le shell courant du processus (y penser avec la commande `tee`).
   e) C'est parti, en utilisant des tubes anonymes (`|`), un tube nommé deux commandes `tee` et `stdbuf`, faire jouer ensemble les deux programmes, pour obtenir, par exemple :

```
499
>
749
<
624
<
561
>
592
<
576
<
568
>
572
<
570
>
571
=
```

4. Maintenant en C. Écrire le programme `nb_secr` avec deux processus : le parent qui fait deviner un nombre choisi au hasard et l'enfant qui le trouve. Vous pouvez utiliser les programmes déjà fournis comme base de travail. Ma solution utilise deux tubes créés avec l'appel système `pipe`. Chaque processus envoie une ligne puis attend une réponse, donc vous pouvez considérer que les messages dans les tubes sont des lignes, pas trop longues. Pour exploiter ces lignes, c'est assez simple dans le cas `<`, `>` et `=`, mais pour les entiers, `sscanf` peut vous aider à associer un entier aux caractères du `buffer` dans lequel écrit `read`.

--- * ---